

تقرير الفحص التقني الشامل

مشروع سهمي (Sahmi) - محلل الأسهم السعودية

تاريخ التقرير: 19 يوليو 2026

آخر Commit مفحوص: 4304bb8 — الفرع: `claude/sahmk-stock-analyzer-telegram-0yykdh`

التقييم العام: 58 / 100

منهجية التقرير: كل بند هنا إما (أ) تم التحقق منه فعليًا بتشغيل أمر حقيقي أو قراءة كود أو سجل حقيقي، أو (ب) مُعَلَّم صراحة "لم يتم التحقق منه". لا توجد افتراضات غير موسومة، ولا تجميل للنتائج.

1. حالة المشروع (Project Status)

السؤال	الإجابة	الدليل
هل المشروع يعمل؟	جزئيًا	محرك التقييم والواجهة يعملان فعليًا. تسليم التنبيهات عبر Telegram في ظروف حقيقية لم يُشاهد ناجحًا ولو مرة واحدة خلال كامل السجل المتوفر.
هل Build ينجح؟	نعم	<code>tsc --noEmit (server)</code> و <code>tsc -b && vite build (frontend)</code> — نُقِّد الآن فعليًا، صفر أخطاء
هل يوجد TypeScript Errors؟	لا - صفر	تشغيل فعلي على كلا المشروعين
هل يوجد Runtime Errors؟	نعم، حقيقية	مسجلة فعليًا في سجلات إنتاج Render خلال آخر 48 ساعة
هل يوجد Console Errors؟	لم يُتحقق	لم يُفتح المتصفح لمراقبة ال Console في هذه الجلسة
هل يوجد Firebase Errors؟	نعم، مؤكدة مرتين	لقطة شاشة فعلية: <code>RESOURCE_EXHAUSTED: Quota exceeded – 100% used, exceeded by 287</code> (19 يوليو)، ونفس الخطأ بتاريخ 18 يوليو

2. حالة الميزات (Features Status)

تصحيح مهم: بعض الميزات أدناه طُننَتْ سابقًا في هذه المحادثة أنها "غير موجودة وتحتاج بناء من الصفر" (عند سؤال المستخدم عن رأيي في تحسينات مقترحة). الفحص الفعلي للكود الآن يُظهر أنها موجودة جزئيًا — تم تصحيح ذلك هنا.

الميزة	الحالة	السبب / التفاصيل	الإنجاز
Investment Score	تعمل فعليًا	6 فحوصات جودة + 4 تقييم + 5 توزيعات + 5 فنية، مطابقة للأوزان 35/25/25/15 — مؤكَّد باختبارات فعلية على رموز حقيقية	100%
استبعاد القطاع المالي	تعمل فعليًا	مؤكَّد فعليًا: سهم 1120 استُبعد بنجاح	100%
Dashboard	جزئية	تعمل، لكن مؤشر "استهلاك API اليوم" معطل فعليًا الآن (قراءة من collection توقفت الكتابة إليه — خطأ #4)	80%
Stock Screener	موجودة فعليًا	Explorer.tsx : بحث + فلتر (قطاع/درجة/Yield/PE/ROE/D-E) + فرز بالنقر على الأعمدة	90%
Dividend Calendar	تعمل فعليًا	موجودة وكاملة في DividendCalendar.tsx ، بفلتر شهر/قطاع	95%
Historical Investment Score	جزئية جدًا	موجود فعليًا: كل تقييم يُضاف تلقائيًا لـ scores/{symbol}/history ، يُعرض كقائمة نصية بدون رسم بياني	50%
Score Change Log	جزئية	نفس آلية البند أعلاه — بدون تفاصيل "أي فحص تغيّر ولماذا"	40%
Confidence Score	جزئية	dataCompleteness محسوب وموجود، معروض كعمود "اكتمال البيانات" — غير مُبرز كـ "درجة ثقة" مستقلة	60%
Risk Score	غير موجودة	صفر كود، صفر واجهة	0%
AI Summary	غير موجودة	—	0%
Dividend Ranking / Top Lists	جزئية	مُتاحة عبر فرز عمود التوزيعات في Explorer، بدون صفحة مخصّصة	40%
Multi-Filter Presets	غير موجودة	فلتر Explorer تُفقد عند تحديث الصفحة (state محلي غير محفوظ)	0%
Event Log	بيانات فقط	syncJobs يُكتَب فعليًا، لكن صفر استهلاك في الواجهة (بحث شامل: لا نتائج)	20%
Opportunity History	غير موجودة	—	0%
Alert Levels	تعمل فعليًا	3 مستويات (79-70 داخلي فقط، 80-89، 90-100) — مؤكَّد بالكود والاختبارات	100%
Scheduled Reports	غير موجودة	تنبيهات فورية فقط	0%
Custom Scoring Weights	تعمل فعليًا	settings.scoringWeights مربوطة بواجهة Settings والباك إند بالكامل	100%

0%	صفر نتائج بحث عن export/xlsx/csv/pdf في الواجهة	غير موجودة	Export Excel / PDF
كود 100% / نتيجة غير مؤكدة	الكود سليم بنيويًا، لكن لم يُؤكَّد نجاح إرسال فعلي حقيقي — راجع القسم 5	غير مؤكدة فعليًا	Telegram Alerts

3. تدقيق Firebase

Collections الموجودة فعليًا (تأكيد شامل عبر grep)

Collection	من يكتب	من يقرأ	ملاحظة
companies	السيرفر	الواجهة + السيرفر	
quotes	السيرفر	الواجهة + السيرفر	
historicalPrices	السيرفر	السيرفر فقط	doc id: {symbol}_{date}
financials	السيرفر	السيرفر فقط	
ratios	السيرفر	الواجهة + السيرفر	
dividends	السيرفر	الواجهة + السيرفر	
scores (+ history)	السيرفر	الواجهة + السيرفر	subcollection معروضة جزئيًا
alerts	السيرفر	الواجهة	
watchlist	العميل مباشرة	الواجهة	لا يوجد route خادم للتحقق
settings	السيرفر	الواجهة + السيرفر	
apiUsage	لا أحد بعد اليوم	الواجهة (معطوبة)	خطأ #4
syncJobs	السيرفر	لا أحد	بيانات ميتة عمليًا
_scheduleState	السيرفر	السيرفر فقط	
_rateLimits	السيرفر	السيرفر فقط	
_devRawResponses	السيرفر (DEV_MODE)	السيرفر فقط	

الإجمالي: 15 مجموعة رئيسية + subcollection واحدة.

Security Rules

- موجودة ومفحوصة فعليًا (95 سطر). القواعد نفسها صحيحة وظيفيًا.
- التعليق التوضيحي قديم/مضلل: يذكر "الكتابة من Cloud Functions" رغم الانتقال الكامل لـ Render منذ commit 6d0757f.
- apiUsage له قاعدة قراءة صريحة رغم توقف الكتابة إليه فعليًا اليوم — قاعدة يتيمة.
- watchlist الوحيد المسموح بكتابة مباشرة من العميل.

Indexes

firestore.indexes.json موجود. لم يتم التحقق من تطابقه الكامل مع كل استعلام مركّب في الكود — لم تظهر أخطاء "index required" فعلية في السجلات المتوفرة.

لا توجد أي Cloud Functions — صفر. الانتقال الكامل لـ Render + GitHub Actions تم في commit `6d0757f`. لا يوجد مجلد `functions/` في المستودع.

Firebase Secrets / بيئة التشغيل

- المفاتيح الحساسة معرّفة في `render.yaml` كـ `sync: false` (تُدخّل يدويًا في Render) — تصميم سليم.
- لم يتم التحقق من القيم الفعلية الحية على Render في هذه الجلسة (لا صلاحية وصول مباشر).
- خدمة الويب على Render نفسها `plan: free` — يفسّر سلوك "spin down" المُشاهد فعليًا.
- `DEV_MODE` معرّف بـ `"false"` في IaC، لكن لم يتم التحقق من القيمة الفعلية الحالية على Render مباشرة.

4. تدقيق SAHMK

السؤال	الإجابة
هل الاتصال يعمل؟	نعم، مؤكّد فعليًا
هل يتم جلب البيانات؟	نعم — شركات، أسعار، بيانات مالية حقيقية جُلبت وحُزّنت فعليًا
كم Endpoint مستخدم؟	7 endpoints حقيقية موثّقة: <code>/companies/</code> ، <code>/company/{symbol}/</code> ، <code>/quotes/</code> ، <code>/historical/{symbol}/</code> ، <code>/financials/{symbol}/</code> ، <code>/dividends/{symbol}/</code> ، <code>/analytics/ratios/{symbol}/</code>
هل يوجد Rate Limit؟	نعم، حقيقي — SAHMK ترجع 429 مع <code>"Expected available in N seconds"</code> ، والكود يحترمها فعليًا (حتى 6 محاولات، حتى 35 ثانية)
هل توجد أخطاء "حد يومي"؟	لا — مؤكّد سابقًا: تجاوزنا 20,000 طلب/يوم أثناء الاختبار بدون أي رفض من SAHMK لسبب "حد يومي"
ما البيانات غير المتوفرة في Starter؟	الجملة <code>/companies/</code> لا ترجع <code>market_id / sector_name</code> إطلاقًا. P/E و P/B غير متوفرين في <code>/analytics/ratios/</code> — مصدرهما الوحيد <code>/company/{symbol}/</code>

5. تدقيق Telegram (تفصيلي)

مسار الكود الكامل (مؤكّد بقراءة الكود سطرًا سطرًا):

- (1) `alerts/evaluateAlert.ts` — 9 شروط بالترتيب، أي شرط يفشل يوقف العملية فورًا (سلوك مقصود).
- (2) `services/telegram/messageBuilder.ts` — بناء نص الرسالة.
- (3) `services/telegram/TelegramService.ts` (سطر 30-94) — طلب HTTP حقيقي لـ `api.telegram.org`.
- (4) `repo/alertsRepo.ts` — حفظ النتيجة بـ `sentToTelegram`.

السؤال	الجواب المحدّد
هل Bot Token موجود؟	يُقرأ من <code>TELEGRAM_BOT_TOKEN</code> . لم يتم التحقق من القيمة الفعلية على <code>Render</code> الآن مباشرة.
هل Chat ID موجود؟	يُقرأ من <code>TELEGRAM_CHAT_ID</code> . لم يتم التحقق من القيمة الفعلية على <code>Render</code> الآن مباشرة.
هل Firebase Function تستدعي Telegram؟	لا يوجد <code>Firebase Function</code> إطلاقًا. الاستدعاء من <code>scanAndAlertAllSymbols()</code> → <code>scheduled/dailyPriceAndAlertScan.ts</code> → <code>evaluateAndMaybeSendAlert()</code>
هل يتم إنشاء التنبيه؟	نعم للمستويين 70-79 و+80 — <code>createAlert()</code> يُستدعى فعليًا بغض النظر عن إرسال Telegram
هل يتم استدعاء Telegram API؟	نعم، لكن فقط عند اجتياز التسعة شروط — لم يحدث فعليًا مع أي رمز حقيقي في السجل المتوفر (كل الاختبارات أظهرت <code>sent: false</code> بسبب "الدرجة أقل من 70")
هل Telegram API يرجع Success؟	لم يتم التحقق منه لسهم حقيقي مستوفي للشروط. الدليل الوحيد المتاح هو زر "اختبار الآن" في Dashboard — ولا يوجد تأكيد أنه ضُفِط ونجح فعليًا
هل تم اختبار إرسال رسالة فعلية؟	لا. لا توجد لقطة شاشة، لا سجل، لا تأكيد نصي من المستخدم بوصول رسالة Telegram حقيقية
أين تتوقف العملية إذا لم تصل الرسالة؟	آخر خطوة مؤكّدة تعمل: حساب الدرجة وتقييم الشروط التسعة (مختبر لرموز حقيقية: 2222، 2280، 7010، 1211 نجحت، 1120 استُبعد صح). أول خطوة غير مؤكّدة: استدعاء <code>sendMessage()</code> الفعلي لسهم حقيقي مستوفي للشروط — لم يحدث قط لأن لا سهم وصل فعليًا لكل الشروط مجتمعة أثناء الاختبار

خلاصة القسم: الكود سليم بنيويًا ومنطقيًا (مؤكّد بالقراءة والاختبارات الوجودية). لكن لا يوجد دليل واحد في تاريخ هذا المشروع على وصول رسالة Telegram حقيقية ناتجة عن تقييم سهم فعلي يستوفي كل الشروط. هذه فجوة حرجية بين "الكود يبدو صحيحًا" و"النظام أثبت أنه يعمل".

6. المهام المجدولة (Scheduled Jobs)

اسم الوظيفة	الغرض	تعمل؟	آخر تشغيل مؤكّد	النتيجة	الخطأ
daily-price-and-alert-scan	أسعار + بيانات تاريخية + تقييم + تنبيهات (يوميًا)	غير مستقرة	Run #24، 19 يوليو، 62 دقيقة	فشل	علق بـ <code>busy:true</code> ساعة كاملة بدون تقدّم — قفل التزامن لا يُحرّر عند التعليق (bug غير مُصلّح)
weekly-financials-scan	القوائم المالية والنسب (أسبوعيًا)	غير مستقرة	Run #24، 19 يوليو، 54 دقيقة	فشل	نفس السبب + استنزاف حصة Firestore (100% مستخدمة، تجاوز 287)
daily-dividends-scan	التوزيعات (يوميًا)	غير مستقرة	Run #24، 19 يوليو، 65 دقيقة	فشل	نفس الأسباب أعلاه

سجل حقيقي (آخر 24 تشغيل، عبر GitHub Actions API فعليًا):

- 24 تشغيل إجمالي منذ 16 يوليو، 10 فاشلة (~42%).
- أسباب مؤقّعة: أخطاء اتصال curl (مُصلّحة)، استنزاف حصة Firestore (مرتين)، انقطاع صامت لجدولة GitHub Actions لساعات (3 مرات على الأقل، خارج نطاق كودنا)، خلل قفل التزامن (اليوم، غير مُصلّح).
- لا يوجد دليل واحد على اكتمال دورة تنبيهات كاملة بنجاح 100% خلال آخر 3 أيام على الأقل.

7. تدقيق Firestore - ما يُحفظ فعليًا

البيانات	تُحفظ؟	التفاصيل
Stocks	✓	companies
Scores	✓	scores/{symbol} — أحدث تقييم فقط
Historical Snapshots	⚠ جزئي	scores/{symbol}/history — لقطة درجة + وقت فقط
Alerts	✓	alerts
Events	⚠ جزئي	syncJobs يُسجّل، لكن بدون واجهة عرض
Reports	✗	غير موجودة
Feed	✗	غير موجودة
Timeline	✗	غير موجود كمفهوم موّحد — مبعثر بين alerts/history/syncJobs

8. قائمة الأخطاء (من الأخطر للأقل)

#1 - قفل التزامن (jobLock) قد يعلق للأبد Critical

الوصف: قفل بالذاكرة أُضيف اليوم لمنع تشغيل نفس المهمة بالتزامن، لكن بدون hard timeout — لو علّقت العملية الأصلية، يبقى القفل محجورًا للأبد.

الملفات: `server/src/routes/internal.ts` , `server/src/utils/jobLock.ts`

التأثير: مؤكّد فعليًا اليوم — `daily-price-and-alert-scan` بقي `busy:true` ساعة كاملة متواصلة. خطر غير مؤكّد: قد يكون القفل لا يزال محجورًا الآن، مما يهدد تشغيله الغد 12:30 — إلا لو أعاد Render تشغيل الخدمة تلقائيًا بسبب الخمول قبل ذلك (لم يُتحقق).

#2 - استنزاف حصة كتابة Firestore اليومية (Spark) Critical

الوصف: حصة 20,000 كتابة/يوم استُنزفت بالكامل مرتين خلال يومين متتاليين (18، 19 يوليو) — مؤكّد بملقطات شاشة فعلية.

السبب: أي دورة اختبار/تشخيص كاملة تستهلك آلاف الكتابات الحقيقية. المشروع لا يزال على خطة Spark المجانية. التأثير: يمنع أي كتابة إضافية حتى التصفير اليومي (~10 صباحًا بتوقيت الرياض).

#3 - عدم موثوقية جدولة GitHub Actions Critical

الوصف: الجدولة التلقائية (كل 30 دقيقة) تتوقف بصمت تام لساعات متعددة (فجوات 3-4+ ساعات مرصودة فعليًا)، بدون أي رسالة خطأ.

السبب: سلوك منصة GitHub Actions نفسها — خارج نطاق كود المشروع تمامًا. التأثير: التنبيه اليومي 12:30 لم يصل فعليًا يومي 18 و19 يوليو بسبب هذا بالضبط.

#4 - مؤشر "استهلاك API اليوم" في Dashboard معطوب فعليًا High

الوصف: بعد نقل عداد الاستخدام للذاكرة (`ed38bae commit`)، توقفت الكتابة لـ `apiUsage` نهائيًا، لكن Dashboard لا يزال يقرأ منها ويعرض `{count}/5000`.

الملفات: `server/src/services/sahmk/rateTracker.ts` , `frontend/src/pages/Dashboard.tsx`

التأثير: معلومة مضلّلة معروضة دائمًا (مجمّدة أو صفر)، والرقم "5000" نفسه كان قديمًا أصلًا.

#5 - عدم التحقق الفعلي من نجاح إرسال Telegram على الإطلاق High

الوصف: راجع القسم 5 — صفر دليل على نجاح إرسال حقيقي.
التأثير: لا ضمان فعلي أن المسار الكامل يعمل حتى لو بدا كل جزء سليمًا على حدة.

#6 - توثيق قديم مضلل في `firestore.rules` Medium

الوصف: يذكر "Cloud Functions" كمصدر الكتابة رغم الانتقال لـ `Render`.
التأثير: لا تأثير وظيفي، يضلّل أي مراجعة مستقبلية.

#7 - بيانات `syncJobs` تُكتب لكن لا تُستهلك Medium

الوصف: كل تشغيلة تكتب تقدّمها، لكن لا واجهة تعرضها (بحث شامل: صفر استخدام).
التأثير: فرصة صائبة لتشخيص مشاكل #1-3 مباشرة من لوحة التحكم.

#8 - `watchlist` بدون تحقق خادم من المدخلات Low

الوصف: كتابة مباشرة من العميل بدون route للتحقق — قرار تصميم مقصود لتطبيق شخصي بمستخدم واحد.

#9 - حزمة الواجهة أكبر من 500KB بدون تقسيم Low

الوصف: تحذير فعلي من `vite build` : 667.67 kB . لا يوجد code splitting.

9. الدين التقني (Technical Debt)

- كود مكرر: لم يُعثر على تكرار واضح — البنية معيارية بوضوح.
- ملفات كبيرة جدًا: لا يوجد ملف يتجاوز 450 سطرًا (الأكثر: `DevTest.tsx` بـ 447 سطرًا).
- `Services/Components` غير مستخدمة: لم يُعثر على أي.
- Dead Code مؤكّد:
 - `getScoreHistory()` في `scoresRepo.ts` — معرّفة، صفر استدعاء في كامل المستودع.
 - مجموعة `apiUsage` بالكامل — تُقرأ لكن لا تُكتب بعد اليوم.
- `TODOs`: صفر — لم يُعثر على أي `TODO/FIXME/HACK/XXX` في كامل قاعدة الكود.
- ملفات تحتاج إعادة هيكلة: لا يوجد ملف واحد يستدعي ذلك عاجلاً.

10. مراجعة المعمارية (Architecture Review)

شجرة المشروع (المستوى الأول والثاني)

```
Sahmi/
├── .github/workflows/scheduled-scans.yml # الجدولة (GitHub Actions cron)
├── firestore.rules / firestore.indexes.json
├── render.yaml # إعدادات نشر (IaC)
├── firebase.json # Hosting فقط - لا Functions
├── server/src/ # Express Backend (Render)
│   ├── alerts/ (2) config/ (2) jobs/ (9)
│   ├── middleware/ (3) repo/ (12) routes/ (2)
│   ├── scheduled/ (3) scoring/ (8) services/sahmk/ (8)
│   ├── services/telegram/ (2) technical/ (5) utils/ (7)
├── frontend/src/ # React + Vite (Firebase Hosting)
│   ├── components/ (5) hooks/ (2) lib/ (2)
│   └── pages/ (8): Login, Dashboard, Explorer, StockDetail,
│       Watchlist, DividendCalendar, Settings, DevTest
```

تدقيق البيانات الفعلي (مؤكّد بالكود)

```
SAHMK API (7 endpoints حقيقية)
├── فقط - المفتاح لا يظهر للعمليات Render سيرفر
└──
    ▼
    services/sahmk/SahmkService.ts + httpClient.ts
    │   ├── retry + تسجيل استخدام بالذاكرة
    │   └──
    │       ▼
    │       jobs/* (refreshQuotes/Historical/FinancialsAndRatios/Dividends/Companies)
    │       ▼
    │       Firestore (repo عبر) ← وسيطة Cloud Functions لا توجد
    │       │   └──
    │       │       └── scoring/* (scores/ يُحسب عند الطلب، يُكتب في)
    │       │       ▼
    │       │       alerts/evaluateAlert.ts (شروط 9)
    │       │       │   └── (فقط لو اجتازت كل الشروط)
    │       │       services/telegram/TelegramService.ts → Telegram Bot API
    │       │       │   └── (Telegram بالتوازي، مستقل عن)
    │       │       Firestore (scores, alerts) → frontend hooks/useCollection.ts
    │       │       │   └── (وسيط route مباشر، بدون onSnapshot)
    │       │       ▼
    │       │       Dashboard / Explorer / StockDetail / DividendCalendar
```

ملاحظة معمارية مهمة: الواجهة لا تمر عبر Render للقراءة — تقرأ من Firestore مباشرة عبر Firebase SDK بمصادقة المستخدم. Render يُستخدم فقط للكتابة (المهام المجدولة) وعمليات خاصة (اختبار Telegram، تحديث الإعدادات، فحص تنبيه يدوي). تصميم سليم يقلّل الحمل على Render.

11. صحة البناء (Build Health)

المقياس	العدد
ملفات السيرفر (ts، بدون اختبارات)	65
ملفات اختبار السيرفر (test.ts.)	6 (29 اختبارًا)
ملفات الواجهة (ts/.tsx.)	21
Components	5
Pages	8
Hooks	2
Repos (طبقة Firestore)	12
Jobs (قابلة للاستدعاء)	9
Scheduled Jobs (مجدولة فعليًا)	3
Cloud Functions	0
Firestore Collections	15 + subcollection واحدة
TypeScript Errors (سيرفر / واجهة)	0 / 0
ESLint Errors (سيرفر / واجهة)	0 / 0 (تحذير واحد فقط)
Runtime Errors مؤكدة (آخر 48 ساعة)	2 نوع مختلف، تكرر عدة مرات
نسبة نجاح آخر 24 تشغيله GitHub Actions	14/24 (~58%)

100 / 58

هذا الرقم يعكس فجوة حقيقية بين جودة الهندسة الأساسية (محرك التقييم، بنية الكود، الواجهة) وموثوقية التشغيل الفعلي (تسليم التنبيهات في وقتها، بدون انقطاع).

ما الذي يمنع المشروع من أن يكون جاهزًا للاستخدام الفعلي؟

الوعد الأساسي: "احسب تقييمًا دقيقًا لكل سهم، وأرسل تنبيهًا فورًا حين يستوفي سهم الشروط." الجزء الأول مؤكّد أنه يعمل. الجزء الثاني لم يُثبت فعليًا حتى الآن — لا يوم واحد موثّق اكتملت فيه الدورة الكاملة بنجاح 100% خلال آخر 3 أيام، ولا دليل واحد على وصول رسالة Telegram حقيقية.

أعلى 10 مشاكل يجب إصلاحها أولًا

1. إضافة hard timeout لقفل `jobLock` حتى لا يعلق للأبد.
2. حسم قرار خطة (Blaze) Firebase لوقف تكرار استنزاف الحصة.
3. إيجاد بديل/تعويض لعدم موثوقية جدولة GitHub Actions.
4. إصلاح أو حذف مؤشر "استهلاك API" المعطوب في Dashboard.
5. تأكيد فعلي واحد على الأقل لوصول رسالة Telegram حقيقية.
6. تحديث تعليق `firestore.rules` القديم.
7. بناء واجهة بسيطة لعرض `syncJobs`.
8. حذف/ربط الكود الميت (`getScoreHistory`).
9. تأكيد القيم الفعلية الحية لمتغيرات البيئة على Render.
10. تقسيم حزمة الواجهة الأمامية (code splitting).

ما الذي يمكن تأجيله

إضافية حقيقية غير موجودة، لكن لا شيء منها يمنع الوظيفة الأساسية من العمل. — ميزات Risk Score, AI Summary, Opportunity History, Scheduled Reports, Export Excel/PDF, Multi-Filter Presets

ما الذي يُعتبر جاهزًا للإنتاج بالفعل

- محرك التقييم (Investment Score) بكل فئاته الأربع — مختبر ومؤكّد.
- استبعاد القطاع المالي — مؤكّد فعليًا.
- Build كامل (سيرفر + واجهة) — نظيف تمامًا، صفر أخطاء.
- الواجهة الأساسية — تعمل، مبنية جيدًا، هوية RTL عربية متسقة.
- تكامل SAHMK — موثوق، يحترم Rate Limiting، بدون بيانات ملقّقة.
- أوزان التقييم المخصصة — كاملة وتعمل من الواجهة للباك إند.